南华大学

计算机学院

实 验 报 告

（ 2026 年秋季学期 ）

课程名称	web 全栈开发
姓　　名	王亨泰
学　　号	20244081108
专　　业	数据科学与大数据技术（中国电子新创班）

一、实验目的

这次实验我用 Vue 3 做了一个完整的前端网站，主题是独立书店「纸间书店」。目标不是把官方教程的示例照抄一遍，而是把模板语法、响应式、组件、路由、表单、过渡这些知识点串起来，做成一个可以真的打开、点击、浏览的网站。

网站没有后台和支付，重点放在页面结构、组件拆分、交互和视觉上。最后完成 4 个主页面和一个 404 页面，共 27 个 Vue 单文件组件。用户可以浏览书目、搜索筛选、加入书架、记录阅读进度、切换夜间模式、查看异步加载的读者短评，也可以填写联系表单。

另外，我把项目部署到了 GitHub Pages，地址是长期有效的。这样交作业时不用只发一个本地压缩包，老师和同学直接点开就能看。

二、实验环境

开发环境比较普通，没有使用额外的 UI 框架或后端服务：

- 系统：Windows 11。
- 运行环境：Node.js v24.21.0，pnpm 11.28.5。
- 框架与工具：Vue 3.5.44、Vite 6.4.5、Vue Router 4.6.4。
- 浏览器与测试：Microsoft Edge 148，Playwright Core 1.64.1。
- 项目目录：paperbook/。开发时运行 pnpm dev，构建用 pnpm build，本地预览用 pnpm preview。

页面样式全部用 CSS 写成，没有引入组件库。书籍封面、书店门面、地图这些插画也是用 CSS 和内联 SVG 画的，所以网站不依赖外部图片，断网时本地预览也不会出现空白图。

三、实验步骤

1. 先定主题和视觉

我没有先写代码，而是先想网站要给人什么感觉。最后选了“独立书店”这个主题：暖纸色背景、森林绿主色、陶土橙强调，标题用宋体，正文用无衬线字体。页面尽量留白，不做好几个大色块，也不做很强烈的动效。

每本书的封面不是找图片，而是由 BookCover.vue 根据颜色和母题动态生成。这样 12 本书的封面风格统一，也不会因为网络问题加载不出来。

2. 搭项目与路由

项目用 Vite 创建，路由使用 Vue Router 的 Hash 模式。这样部署到 GitHub Pages 的子目录时，刷新 /#/shelf 或 /#/books/siddhartha 不会 404。四个主页面分别是首页、书架、书籍详情和关于，另外加了一个自定义 404。

3. 拆分组件

组件大致按“页面—业务组件—基础组件”三层来分。页面只负责组织区块；BookCard、BookCover、BookTabs、ReadingProgress 这些是业务组件；BaseButton、BaseTag、SectionTitle、EmptyState 是基础组件。最后统计下来是 27 个 .vue 文件

4. 做交互和状态

书架状态放在 useShelf() 里，用 reactive 保存 ids 和 progress，用 computed 算数量、平均进度和读完数量，再用 watch 深度监听并写入 localStorage。刷新页面后书架和阅读进度不会丢。搜索、筛选、排序是书架页自己的状态，通过 URL query 同步，复制链接给别人也能还原筛选条件。

详情页的读者短评用 defineAsyncComponent + Suspense 做成异步组件，打开标签时先显示加载提示，再显示内容和发布表单。夜间模式通过 data-theme 切换 CSS 变量，不写第二套样式。

5. 响应式和部署

桌面端是左右分栏和四列书卡，平板端把工具栏改成上下布局，手机端把导航收进抽屉、书卡改单列。构建时 base 设为 ./，资源用相对路径；GitHub Pages 用 gh-pages 分支托管 dist/ 构建产物。

四、代码示例

下面只放三段比较关键的代码，分别对应路由缓存、书架持久化和异步短评。

```
// src/router/index.js（节选）
const router = createRouter({
  history: createWebHashHistory(),
  routes: [
    { path: '/', component: () => import('../views/HomeView.vue') },
    { path: '/shelf', component: () => import('../views/ShelfView.vue') },
    { path: '/books/:id', props: true,
      component: () => import('../views/BookDetailView.vue'),
      beforeEnter: (to) => getBook(to.params.id) || { name: 'not-found' } },
  ],
  scrollBehavior: () => ({ top: 0 }),
})
```

代码 1　路由表、动态参数和 Hash 模式

```
// src/composables/useShelf.js（节选）
const state = reactive({ ids: saved.ids, progress: saved.progress })
const count = computed(() => state.ids.length)

watch(
  () => ({ ids: [...state.ids], progress: { ...state.progress } }),
  (value) => localStorage.setItem(KEY, JSON.stringify(value)),
  { deep: true },
)

function toggle(id) {
  return state.ids.includes(id) ? remove(id) : add(id)
}
```

代码 2　书架状态和 localStorage 持久化

```
<!-- src/views/BookDetailView.vue（节选） -->
<Suspense>
  <BookReviews :book="book" />
  <template #fallback>
    <p>正在把读者的便签取下来…</p>
  </template>
</Suspense>
```

代码 3　异步短评和 Suspense 占位

代码没有做过度封装，重点是把状态放在合适的位置。比如书架状态只有一份，搜索和筛选只属于书架页，这样组件之间不会互相打架。

五、实验结果

网站地址：https://w123-123-5.github.io/paperbook/

这个地址是 GitHub Pages 的长期地址，打开就是首页。顶部导航可以进入书架、关于；书架可以搜索、按类别和气质筛选、切换网格和列表；详情页可以加入书架、拖动阅读进度、查看异步短评；关于页有 FAQ 和联系表单。手机端也能正常打开，导航会收进抽屉。

最终 pnpm build 构建通过，Playwright 跑了 12 项交互检查，覆盖首页、加入书架、搜索、筛选、列表视图、详情页、阅读进度、FAQ、联系表单、404、移动菜单和夜间模式，控制台没有报错。



图 1　首页首屏：书堆、主标题和本周共读

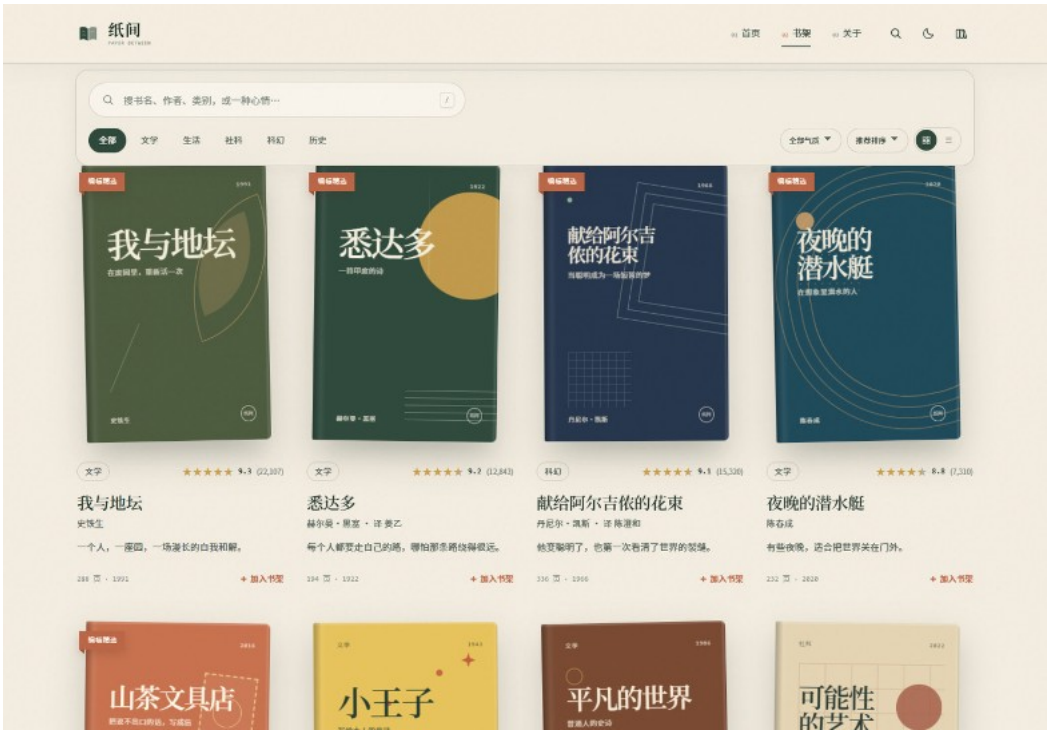


图 2　书架页：搜索、筛选、排序和加入书架



图 3　书籍详情：封面、书摘、借阅和阅读进度



图 4　关于页：书店门面插画、数据和故事入口



图 5　手机端首页：单列布局和收起后的导航

六、问题与解决

1. 刷新子页面 404

一开始用 History 路由，部署到静态托管后刷新 /shelf 会 404。后来改成 Hash 路由，构建时用相对路径 ./，GitHub Pages 子目录也不需要额外的服务器重写。

2. 清空搜索后关键词又回来

搜索框清空后，过一会儿旧关键词又被写回去。查下来是 RouterView 用 fullPath 做 key，query 一变就重建整个书架页，旧实例的路由监听又把值写回。把 key 改成 route.path 后，query 只更新状态，不再重建页面。

3. 书架刷新后丢失

最初书架只存在内存里，刷新就没了。后来用 reactive 保存 ids 和 progress，用 computed 派生统计，用深度 watch 写入 localStorage，并在初始化时过滤无效 id，问题解决。

4. Suspense 的加载提示看不到

异步组件加载太快，fallback 一闪而过。给 BookReviews 顶层加了一个 520 ms 的 await，让加载状态真实显示出来，也方便截图。

5. 长书名把封面撑破

《献给阿尔吉侬的花束》这类长书名在固定字号下会溢出。BookCover 根据标题长度计算 --title-size，再用容器查询单位调整字号，短书名和长书名都能放下。

七、实验总结

这次实验最大的收获不是记住了多少 API，而是理解了组件边界和数据流。最开始我把很多状态都放在页面里，组件之间靠事件一层层往上抛，越写越乱；后来把书架、主题、Toast 抽成组合式函数，用 provide/inject 注入，页面和组件的职责就清楚了。

另一个体会是，路由不只是“切换页面”。query、key、KeepAlive 和滚动行为都会影响用户对“返回”的感受。把 key 从 fullPath 改成 path、让 query 只做状态来源之后，搜索和筛选才真正稳定下来。

网站还有不少可以改进的地方：数据是静态的，没有真实账号和借阅记录；搜索只做简单的包含匹配；移动端菜单的焦点管理还可以更细。不过作为 Vue 练习，它已经把我这学期学到的内容串成了一个能用的作品。

最后把网站部署到 GitHub Pages，看到别人能直接打开链接、手机上的布局也正常，是这次实验最有成就感的部分。